PYTHON ACTIVITY-BASED LEARNING

FULL PROJECT REPORT

10 Selected Python Case Studies

Student Name	Tadikamalla Devi Sri Naga Pravallika
Roll Number	26001499
Branch	CSD
Section	2

Prepared from the assigned Python Activity-Based Learning case-study list. Each case study includes a problem statement, objectives, assumptions, algorithm, text flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

Selected Case Studies: 11, 12, 13, 14, 18, 26, 29, 35, 37 and 38

Contents

Case Study	Title
11	Examination Result
12	Attendance Recovery
13	ATM Transaction Simulation
14	Warehouse Inventory Reconciliation
18	Fuel Consumption and Route Comparison
26	Examination Seating Generator
29	Password Validation
35	Text Compression Using Character Runs
37	Calendar Generator
38	ATM Menu-Driven Banking Simulation

Note: Flowcharts are provided as clear text-arrow flowcharts so they remain readable in a text-based report. The output section for each case study is a sample console output; run the code to capture actual screenshots if your faculty specifically requires screenshots.

Case Study 11: Examination Result

1. Problem Statement

Write a program to read a student's marks in three subjects, calculate total marks and percentage, and decide whether the student passes. The student passes only when each subject mark is at least 35.

2. Objectives

• Read three subject marks from the keyboard.

• Calculate total marks and percentage.

• Use a conditional statement to display PASS or FAIL.

3. Assumptions

• Each subject is marked out of 100.

• Marks are whole numbers from 0 to 100.

• A student passes only if all three subject marks are 35 or more.

4. Algorithm

1. Start.

2. Read marks m1, m2, and m3.

3. If any mark is outside 0–100, display an invalid-marks message and stop.

4. Calculate total = m1 + m2 + m3.

5. Calculate percentage = total / 3.

6. If all three marks are at least 35, display PASS; otherwise display FAIL.

7. Display total and percentage.

8. Stop.

5. Flowchart

Start → Input three marks → Validate marks → Calculate total and percentage → Are all marks ≥ 35? → Yes: PASS / No: FAIL → Display result → End

6. Pseudocode

```
BEGIN
  INPUT m1, m2, m3
  IF any mark < 0 OR any mark > 100 THEN
      PRINT "Invalid marks"
  ELSE
      total ← m1 + m2 + m3
      percentage ← total / 3
      IF m1 >= 35 AND m2 >= 35 AND m3 >= 35 THEN
          PRINT "PASS"
      ELSE
          PRINT "FAIL"
      ENDIF
      PRINT total, percentage
  ENDIF
END
```

7. Python Source Code

```
m1 = int(input("Enter marks in Subject 1: "))
m2 = int(input("Enter marks in Subject 2: "))
m3 = int(input("Enter marks in Subject 3: "))

if not (0 <= m1 <= 100 and 0 <= m2 <= 100 and 0 <= m3 <= 100):
    print("Invalid marks")
else:
    total = m1 + m2 + m3
    percentage = total / 3

    if m1 >= 35 and m2 >= 35 and m3 >= 35:
        result = "PASS"
```

```
else:
    result = "FAIL"

print("Total Marks:", total)
print("Percentage:", round(percentage, 2))
print("Result:", result)
```

8. Test Cases

Input / Test	Expected Result
80, 75, 90	Total 245; Percentage 81.67; PASS
80, 20, 90	Total 190; Percentage 63.33; FAIL
100, 100, 100	Total 300; Percentage 100.00; PASS
-5, 80, 70	Invalid marks

9. Sample Output (Console Screenshot Reference)

```
Enter marks in Subject 1: 80
Enter marks in Subject 2: 75
Enter marks in Subject 3: 90
Total Marks: 245
Percentage: 81.67
Result: PASS
```

10. Conclusion

The program calculates a student's total and percentage and applies a subject-wise passing rule using simple arithmetic and conditional statements.

Case Study 12: Attendance Recovery

1. Problem Statement

Write a program to calculate current attendance percentage and find the minimum number of consecutive classes a student must attend to reach a 75% attendance threshold.

2. Objectives

• Calculate attendance percentage.

• Check whether the student already meets the 75% threshold.

• Use a loop to find the minimum additional classes required.

3. Assumptions

• Classes attended cannot be negative or exceed total classes.

• Total classes must be greater than zero.

• The student attends every additional class.

4. Algorithm

1. Start.

2. Input attended classes and total classes.

3. Validate total > 0 and 0 ≤ attended ≤ total.

4. Calculate percentage = attended / total × 100.

5. If percentage is at least 75, display that the student is eligible.

6. Otherwise, set required = 0 and repeat increasing required by one until (attended + required) / (total + required) × 100 is at least 75.

7. Display percentage and required classes.

8. Stop.

5. Flowchart

Start → Input attended and total → Validate input → Calculate percentage → Is attendance ≥ 75%? → Yes: Eligible / No: Loop until threshold reached → Display answer → End

6. Pseudocode

```
BEGIN
  INPUT attended, total
  IF total <= 0 OR attended < 0 OR attended > total THEN
      PRINT "Invalid input"
  ELSE
      percentage ← attended / total * 100
      IF percentage >= 75 THEN
          PRINT "Attendance Eligible"
      ELSE
          required ← 0
          WHILE (attended + required) / (total + required) * 100 < 75
              required ← required + 1
          ENDWHILE
          PRINT required
      ENDIF
  ENDIF
END
```

7. Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    percentage = attended / total * 100
    print("Attendance Percentage:", round(percentage, 2))
```

```
if percentage >= 75:
    print("Attendance Eligible")
else:
    required = 0
    while (attended + required) / (total + required) * 100 < 75:
        required += 1
    print("Additional classes required:", required)
```

8. Test Cases

Input / Test	Expected Result
60, 100	Attendance 60.00%; 60 additional classes
80, 100	Attendance 80.00%; Attendance Eligible
0, 10	Attendance 0.00%; 30 additional classes
5, 0	Invalid input

9. Sample Output (Console Screenshot Reference)

```
Enter classes attended: 60
Enter total classes: 100
Attendance Percentage: 60.0
Additional classes required: 60
```

10. Conclusion

The program combines percentage calculation and a while loop to determine the minimum future classes needed to meet an attendance threshold.

Case Study 13: ATM Transaction Simulation

1. Problem Statement

Create a simple ATM simulation that performs one deposit, withdrawal, or balance inquiry and displays the final balance.

2. Objectives

• Provide a menu of ATM operations.

• Update the balance after valid deposits and withdrawals.

• Reject non-positive amounts and withdrawals above the available balance.

3. Assumptions

• The starting balance is Rs. 5,000.

• Only one transaction is performed each time the program runs.

• Amounts are entered as whole rupees.

4. Algorithm

1. Start with balance = 5000.

2. Display Deposit, Withdraw, and Check Balance options.

3. Read the user's choice.

4. If Deposit, read a positive amount and add it to balance.

5. If Withdraw, read an amount and subtract it only if positive and not greater than balance.

6. If Check Balance, display the balance.

7. For any other choice, display Invalid Choice.

8. Display final balance and stop.

5. Flowchart

Start → Set balance → Display menu → Read choice → Deposit / Withdraw / Check balance / Invalid choice → Display final balance → End

6. Pseudocode

```
BEGIN
  balance ← 5000
  DISPLAY menu
  INPUT choice
  IF choice = 1 THEN
      INPUT amount
      IF amount > 0 THEN balance ← balance + amount
      ELSE PRINT "Invalid Amount"
  ELSE IF choice = 2 THEN
      INPUT amount
      IF amount > 0 AND amount <= balance THEN balance ← balance - amount
      ELSE PRINT "Invalid Amount or Insufficient Balance"
  ELSE IF choice = 3 THEN
      PRINT balance
  ELSE
      PRINT "Invalid Choice"
  ENDIF
  PRINT balance
END
```

7. Python Source Code

```
balance = 5000

print("1. Deposit")
print("2. Withdraw")
print("3. Check Balance")
choice = int(input("Enter your choice: "))

if choice == 1:
    amount = int(input("Enter deposit amount: "))
```

```
    if amount > 0:
        balance += amount
        print("Deposit Successful")
    else:
        print("Invalid Amount")
elif choice == 2:
    amount = int(input("Enter withdrawal amount: "))
    if amount > 0 and amount <= balance:
        balance -= amount
        print("Withdrawal Successful")
    else:
        print("Invalid Amount or Insufficient Balance")
elif choice == 3:
    print("Balance:", balance)
else:
    print("Invalid Choice")

print("Final Balance:", balance)
```

8. Test Cases

Input / Test	Expected Result
Choice 1; amount 1000	Deposit Successful; Final Balance 6000
Choice 2; amount 1500	Withdrawal Successful; Final Balance 3500
Choice 2; amount 6000	Insufficient balance; Final Balance 5000
Choice 3	Balance 5000

9. Sample Output (Console Screenshot Reference)

```
1. Deposit
2. Withdraw
3. Check Balance
Enter your choice: 1
Enter deposit amount: 1000
Deposit Successful
Final Balance: 6000
```

10. Conclusion

The simulation demonstrates menu selection, conditional logic, and safe balance updates for basic ATM transactions.

Case Study 14: Warehouse Inventory Reconciliation

1. Problem Statement

Calculate closing stock for several products using opening stock, received quantities, and dispatched quantities, then display each product's closing stock.

2. Objectives

• Store product names and quantities in lists.

• Calculate closing stock using a simple formula.

• Display a product-wise inventory report.

3. Assumptions

• All quantity lists have the same length.

• Opening, received, and dispatched quantities are non-negative.

• Closing stock is opening stock + received stock – dispatched stock.

4. Algorithm

1. Start.

2. Create lists for product names, opening stock, received stock, and dispatched stock.

3. Loop through each product index.

4. Calculate closing = opening + received – dispatched.

5. Display product name and closing stock.

6. Stop.

5. Flowchart

Start → Create product and stock lists → Loop through products → Calculate closing stock → Display product result → More products? → Yes: repeat / No: End

6. Pseudocode

```
BEGIN
  products ← ["Pen", "Book", "Pencil"]
  opening ← [100, 50, 80]
  received ← [20, 30, 10]
  dispatched ← [40, 15, 25]
  FOR each index i
      closing ← opening[i] + received[i] - dispatched[i]
      PRINT products[i], closing
  ENDFOR
END
```

7. Python Source Code

```
products = ["Pen", "Book", "Pencil"]
opening = [100, 50, 80]
received = [20, 30, 10]
dispatched = [40, 15, 25]

for i in range(len(products)):
    closing = opening[i] + received[i] - dispatched[i]
    print("Product:", products[i])
    print("Closing Stock:", closing)
    print()
```

8. Test Cases

Input / Test	Expected Result
Pen: 100, 20, 40	Closing stock = 80
Book: 50, 30, 15	Closing stock = 65

Input / Test	Expected Result
Pencil: 80, 10, 25	Closing stock = 65
Opening 10, received 0, dispatched 12	Closing stock = -2 (signals inconsistent stock data)

9. Sample Output (Console Screenshot Reference)

```
Product: Pen
Closing Stock: 80

Product: Book
Closing Stock: 65

Product: Pencil
Closing Stock: 65
```

10. Conclusion

The program uses parallel lists and a loop to calculate and report closing stock for multiple products.

Case Study 18: Fuel Consumption and Route Comparison

1. Problem Statement

Calculate the fuel required and fuel cost for a journey using the distance, vehicle mileage, and fuel price supplied by the user.

2. Objectives

• Read journey distance, mileage, and price per litre.

• Calculate fuel required and total cost.

• Validate that all inputs are positive.

3. Assumptions

• Distance is measured in kilometres.

• Mileage is measured in kilometres per litre.

• Fuel price is supplied in rupees per litre.

4. Algorithm

1. Start.

2. Input distance, mileage, and fuel price.

3. If any input is not positive, display Invalid Input.

4. Otherwise calculate fuel = distance / mileage.

5. Calculate cost = fuel × price.

6. Display fuel required and total cost rounded to two decimal places.

7. Stop.

5. Flowchart

Start → Input distance, mileage, price → Are all positive? → No: Invalid input / Yes: Calculate fuel and cost → Display results → End

6. Pseudocode

```
BEGIN
  INPUT distance, mileage, price
  IF distance > 0 AND mileage > 0 AND price > 0 THEN
      fuel ← distance / mileage
      cost ← fuel * price
      PRINT fuel, cost
  ELSE
      PRINT "Invalid Input"
  ENDIF
END
```

7. Python Source Code

```
distance = float(input("Enter journey distance in km: "))
mileage = float(input("Enter vehicle mileage in km/litre: "))
price = float(input("Enter fuel price per litre: "))

if distance > 0 and mileage > 0 and price > 0:
    fuel = distance / mileage
    cost = fuel * price
    print("Fuel Required:", round(fuel, 2), "litres")
    print("Total Fuel Cost: Rs.", round(cost, 2))
else:
    print("Invalid Input")
```

8. Test Cases

Input / Test	Expected Result
200 km, 20 km/l, Rs. 105/l	Fuel 10.00 litres; cost Rs. 1050.00

Input / Test	Expected Result
150 km, 15 km/l, Rs. 100/l	Fuel 10.00 litres; cost Rs. 1000.00
0 km, 20 km/l, Rs. 105/l	Invalid Input
200 km, 0 km/l, Rs. 105/l	Invalid Input

9. Sample Output (Console Screenshot Reference)

```
Enter journey distance in km: 200
Enter vehicle mileage in km/litre: 20
Enter fuel price per litre: 105
Fuel Required: 10.0 litres
Total Fuel Cost: Rs. 1050.0
```

10. Conclusion

The program applies arithmetic formulas and input validation to estimate journey fuel requirements and cost.

Case Study 26: Examination Seating Generator

1. Problem Statement

Generate sequential seat identifiers for a classroom using the number of rows and columns entered by the user.

2. Objectives

• Accept row and column counts.

• Use nested loops to generate seat labels.

• Display seats in a row-and-column layout.

3. Assumptions

• Rows and columns must both be positive integers.

• Seats are labelled sequentially from S1.

• No reserved-seat or alternate-seating rules are applied in this simplified version.

4. Algorithm

1. Start.

2. Input number of rows and columns.

3. If either value is not positive, display Invalid dimensions.

4. Otherwise set seat = 1.

5. Use an outer loop for each row and an inner loop for each column.

6. Print the current seat label and increase seat by one.

7. After each row, print a new line.

8. Stop.

5. Flowchart

Start → Input rows and columns → Valid dimensions? → No: Invalid dimensions / Yes: Set seat = 1 → Nested loops print seats → End

6. Pseudocode

```
BEGIN
  INPUT rows, cols
  IF rows <= 0 OR cols <= 0 THEN
      PRINT "Invalid dimensions"
  ELSE
      seat ← 1
      FOR i FROM 1 TO rows
          FOR j FROM 1 TO cols
              PRINT "S" + seat
              seat ← seat + 1
          ENDFOR
          PRINT newline
      ENDFOR
  ENDIF
END
```

7. Python Source Code

```
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

if rows <= 0 or cols <= 0:
    print("Invalid dimensions")
else:
    seat = 1
    for i in range(rows):
        for j in range(cols):
            print("S" + str(seat), end="\t")
            seat += 1
        print()
```

8. Test Cases

Input / Test	Expected Result
Rows 3, columns 4	12 seats displayed from S1 to S12 in 3 rows
Rows 1, columns 2	S1 S2
Rows 0, columns 4	Invalid dimensions
Rows 2, columns -1	Invalid dimensions

9. Sample Output (Console Screenshot Reference)

```
Enter number of rows: 3
Enter number of columns: 4
S1      S2      S3      S4
S5      S6      S7      S8
S9      S10     S11     S12
```

10. Conclusion

The program uses nested loops to produce a simple classroom seating layout with sequential seat identifiers.

Case Study 29: Password Validation

1. Problem Statement

Check whether a password contains at least eight characters, an uppercase letter, a lowercase letter, and a digit.

2. Objectives

• Read a password string.

• Use string methods to check character categories.

• Display whether the password satisfies the specified basic rules.

3. Assumptions

• The minimum password length is eight characters.

• At least one uppercase letter, one lowercase letter, and one digit are required.

• Special characters are optional in this simplified version.

4. Algorithm

1. Start.

2. Input the password.

3. Set upper, lower, and digit flags to False.

4. Loop through each character and update the relevant flag.

5. If length is at least eight and all three flags are True, display Valid Password.

6. Otherwise display Invalid Password and the requirements.

7. Stop.

5. Flowchart

Start → Input password → Check each character → Check length and character flags → All rules satisfied? → Yes: Valid / No: Invalid → End

6. Pseudocode

```
BEGIN
  INPUT password
  upper ← False; lower ← False; digit ← False
  FOR each character ch in password
      IF ch is uppercase THEN upper ← True
      IF ch is lowercase THEN lower ← True
      IF ch is a digit THEN digit ← True
  ENDFOR
  IF length(password) >= 8 AND upper AND lower AND digit THEN
      PRINT "Valid Password"
  ELSE
      PRINT "Invalid Password"
  ENDIF
END
```

7. Python Source Code

```
password = input("Enter password: ")

upper = False
lower = False
digit = False

for ch in password:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True

if len(password) >= 8 and upper and lower and digit:
    print("Valid Password")
```

```
else:
    print("Invalid Password")
    print("Use at least 8 characters, one uppercase,")
    print("one lowercase letter, and one digit.")
```

8. Test Cases

Input / Test	Expected Result
Hello123	Valid Password
hello123	Invalid Password (no uppercase letter)
HELLO123	Invalid Password (no lowercase letter)
HelloWorld	Invalid Password (no digit)
Hi12	Invalid Password (too short)

9. Sample Output (Console Screenshot Reference)

```
Enter password: Hello123
Valid Password
```

10. Conclusion

The program demonstrates string iteration, character-testing methods, Boolean flags, and conditional validation.

Case Study 35: Text Compression Using Character Runs

1. Problem Statement

Compress a string by replacing each group of consecutive repeated characters with the character followed by the number of repetitions.

2. Objectives

• Read a string.

• Identify consecutive repeated characters using a loop.

• Create and display a compressed representation.

3. Assumptions

• Character case is preserved.

• Each run is encoded as character plus count, including runs of length one.

• This is a simple run-length encoding exercise, not a secure or general-purpose compression tool.

4. Algorithm

1. Start.

2. Input text and initialize an empty result string.

3. If the input is empty, display an empty compressed string.

4. Otherwise initialize count = 1.

5. Loop through characters from the second character onward.

6. If the current character equals the previous character, increase count.

7. Otherwise append the previous character and count to the result, then reset count to one.

8. Append the final character and count.

9. Display original and compressed text.

10. Stop.

5. Flowchart

Start → Input text → Initialize result and count → Compare adjacent characters → Same: increase count / Different: append run and reset count → Append final run → Display result → End

6. Pseudocode

```
BEGIN
  INPUT text
  result ← empty string
  IF text is not empty THEN
      count ← 1
      FOR i FROM 1 TO length(text)-1
          IF text[i] = text[i-1] THEN
              count ← count + 1
          ELSE
              result ← result + text[i-1] + count
              count ← 1
          ENDIF
      ENDFOR
      result ← result + last character + count
  ENDIF
  PRINT result
END
```

7. Python Source Code

```
text = input("Enter a string: ")
result = ""

if text:
    count = 1
    for i in range(1, len(text)):
```

```
        if text[i] == text[i - 1]:
            count += 1
        else:
            result += text[i - 1] + str(count)
            count = 1

    result += text[-1] + str(count)

print("Original Text:", text)
print("Compressed Text:", result)
```

8. Test Cases

Input / Test	Expected Result
aaabbcccc	a3b2c4
abcd	a1b1c1d1
aaaa	a4
(empty string)	Compressed Text is empty

9. Sample Output (Console Screenshot Reference)

```
Enter a string: aaabbcccc
Original Text: aaabbcccc
Compressed Text: a3b2c4
```

10. Conclusion

The program uses string indexing and loops to identify repeated character runs and build a simple compressed string.

Case Study 37: Calendar Generator

1. Problem Statement

Display a simple monthly calendar when the user supplies the number of days in the month and the weekday on which the month begins.

2. Objectives

• Accept the number of days and starting weekday.

• Use loops to align the first date and print all dates.

• Format dates into seven-day rows.

3. Assumptions

• Starting weekday uses 0 for Monday through 6 for Sunday.

• The month length is 28, 29, 30, or 31 days.

• The program prints a simplified calendar without month names.

4. Algorithm

1. Start.

2. Input days and starting weekday.

3. Validate that days is 28–31 and starting weekday is 0–6.

4. Print weekday headings.

5. Print blank spaces for the days before the first date.

6. Loop from day 1 to days.

7. Print each date and start a new line at the end of each week.

8. Print a final newline and stop.

5. Flowchart

Start → Input days and start weekday → Validate → Print weekday headings → Print initial blanks → Loop through dates → New line after each week → End

6. Pseudocode

```
BEGIN
  INPUT days, start
  IF days < 28 OR days > 31 OR start < 0 OR start > 6 THEN
      PRINT "Invalid input"
  ELSE
      PRINT weekday headings
      FOR i FROM 0 TO start-1
          PRINT blank cell
      ENDFOR
      FOR day FROM 1 TO days
          PRINT day in a 3-character field
          IF (start + day) MOD 7 = 0 THEN PRINT newline
      ENDFOR
      PRINT newline
  ENDIF
END
```

7. Python Source Code

```
days = int(input("Enter number of days in the month (28-31): "))
start = int(input("Enter starting weekday (0=Mon, 6=Sun): "))

if days < 28 or days > 31 or start < 0 or start > 6:
    print("Invalid input")
else:
    print("Mon Tue Wed Thu Fri Sat Sun")

    for i in range(start):
        print("    ", end="")
```

```
    for day in range(1, days + 1):
        print("%3d " % day, end="")
        if (start + day) % 7 == 0:
            print()

    print()
```

8. Test Cases

Input / Test	Expected Result
30 days, start 3	Dates 1–30 shown, starting under Thursday
28 days, start 0	Dates 1–28 shown, starting under Monday
31 days, start 6	Dates 1–31 shown, starting under Sunday
32 days, start 2	Invalid input
30 days, start 7	Invalid input

9. Sample Output (Console Screenshot Reference)

```
Enter number of days in the month (28-31): 30
Enter starting weekday (0=Mon, 6=Sun): 3
Mon Tue Wed Thu Fri Sat Sun
              1   2   3   4
  5   6   7   8   9  10  11
 12  13  14  15  16  17  18
 19  20  21  22  23  24  25
 26  27  28  29  30
```

10. Conclusion

The program demonstrates nested formatting logic, loops, and weekday alignment to generate a basic text calendar.

Case Study 38: ATM Menu-Driven Banking Simulation

1. Problem Statement

Create a menu-driven ATM program that repeatedly supports deposits, withdrawals, balance inquiries, and exit until the user chooses to leave.

2. Objectives

• Use a while loop to repeat the menu.

• Update the account balance after valid transactions.

• Handle invalid choices and amounts.

3. Assumptions

• The initial balance is Rs. 5,000.

• Transaction amounts are whole rupees.

• Deposits and withdrawals must be positive; withdrawals cannot exceed the balance.

4. Algorithm

1. Start with balance = 5000.

2. Repeat while the user has not chosen Exit.

3. Display the menu and read a choice.

4. If Deposit, read a positive amount and add it to balance.

5. If Withdraw, read a positive amount that does not exceed the balance and subtract it.

6. If Check Balance, display the balance.

7. If Exit, display a thank-you message and stop the loop.

8. Otherwise display Invalid Choice.

9. Stop.

5. Flowchart

Start → Set balance → Display menu → Read choice → Deposit / Withdraw / Balance / Exit / Invalid → Return to menu unless Exit → End

6. Pseudocode

```
BEGIN
  balance ← 5000
  WHILE True
      DISPLAY menu
      INPUT choice
      IF choice = 1 THEN
          INPUT amount
          IF amount > 0 THEN balance ← balance + amount
          ELSE PRINT "Invalid Amount"
      ELSE IF choice = 2 THEN
          INPUT amount
          IF amount > 0 AND amount <= balance THEN balance ← balance - amount
          ELSE PRINT "Invalid Amount or Insufficient Balance"
      ELSE IF choice = 3 THEN PRINT balance
      ELSE IF choice = 4 THEN PRINT "Thank you"; BREAK
      ELSE PRINT "Invalid Choice"
      ENDIF
  ENDWHILE
END
```

7. Python Source Code

```
balance = 5000

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
```

```
print("4. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    amount = int(input("Enter deposit amount: "))
    if amount > 0:
        balance += amount
        print("Deposit Successful")
    else:
        print("Invalid Amount")

elif choice == 2:
    amount = int(input("Enter withdrawal amount: "))
    if amount > 0 and amount <= balance:
        balance -= amount
        print("Withdrawal Successful")
    else:
        print("Invalid Amount or Insufficient Balance")

elif choice == 3:
    print("Current Balance:", balance)

elif choice == 4:
    print("Thank you for using ATM")
    break

else:
    print("Invalid Choice")
```

8. Test Cases

Input / Test	Expected Result
Choice 1; amount 1000; then choice 3; then 4	Balance becomes 6000; inquiry shows 6000; exits
Choice 2; amount 1200; then 4	Balance becomes 3800; exits
Choice 2; amount 7000	Withdrawal rejected; balance remains 5000
Choice 9	Invalid Choice

9. Sample Output (Console Screenshot Reference)

```
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 1
Enter deposit amount: 1000
Deposit Successful

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 3
Current Balance: 6000

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 4
Thank you for using ATM
```

10. Conclusion

The program combines loops and conditional statements to create a simple interactive ATM menu that supports repeated transactions.